

Consistency Models

Phase 2 · Intermediate · Estimated time: 1.5 hours

1. Concept From First Principles

Chapter 1 introduced replication lag as a side effect of read replicas. This chapter asks the deeper question directly: once your data lives in more than one place, what guarantee do you actually give users about seeing the latest write? That guarantee is called a **consistency model**, and choosing one is one of the most consequential decisions in any distributed system's design.

Strong consistency guarantees that once a write completes, every subsequent read — from any replica, anywhere — returns that write's value or something newer. There is never a window where a client can see stale data. This is the mental model most backend engineers default to, because it's how a single, non-replicated database naturally behaves. Achieving it across multiple replicas, however, requires coordination on every write — typically waiting for a quorum of replicas to acknowledge before confirming the write to the client — which directly costs latency and availability during network partitions, a tension Chapter 19's CAP theorem formalizes precisely.

Eventual consistency relaxes this guarantee: after a write, replicas will *eventually* converge to the same value, but there's no bound on exactly when, and different clients reading different replicas at the same moment may see different answers. In exchange, eventually consistent systems can acknowledge writes faster (no need to wait for every replica) and stay available even when some replicas are unreachable. A social media like-count or view-count is the classic example: it's fine if it takes a few seconds to reflect everywhere, and the system staying responsive matters more than split-second accuracy on a number nobody is making a financial decision based on.

Backend engineer analogy

This is the same trade-off as choosing between a synchronous **DB::transaction()** that blocks until committed everywhere it needs to be, versus dispatching an async job via Laravel's queue (or a Node.js message emitter) that updates a read-optimized cache "whenever it gets to it." You already reach for the async version for things like updating a search index or a recommendation cache after a write — that's a deliberate, everyday choice of eventual consistency over strong consistency.

The middle ground: causal and read-your-writes consistency

Between the two extremes sit weaker-than-strong-but-stronger-than-eventual models that solve specific practical problems. **Read-your-writes consistency** guarantees a specific user always sees their own recent writes, even if other users might briefly see stale data — solving exactly the profile-update bug from Chapter 1. **Causal consistency** guarantees that operations which are causally related (a reply to a comment must never appear before the comment it replies to) are seen in the correct order by everyone, even though unrelated operations can still be seen in different orders by different observers.

Choosing a consistency model is not a one-time, whole-system decision — mature systems mix models deliberately, per data type. A payments ledger balance needs strong consistency (you cannot show a different account balance to different readers). A product's "people also bought" recommendation widget can be eventually consistent for hours without anyone noticing or caring. The interview skill being tested is rarely "do you know the definitions" — it's whether you can look at a specific field or feature and correctly argue which consistency model it actually needs, rather than defaulting to strong consistency everywhere out of caution, which quietly taxes every read and write in the system.

It also helps to separate consistency (this chapter) from availability (Chapter 2) as genuinely different axes, even though they interact. A system can be strongly consistent and highly available in the common case, and only be forced to choose between the two during an actual network partition — which is precisely the scenario the CAP theorem in Chapter 19 is built to reason about.

2. Real-World Example / Case Study

Razorpay's and PhonePe's ledger and balance systems demand strong consistency — two different reads of the same account balance, moments apart, must never disagree, and a payment confirmation must be durable and visible everywhere before the user is told the payment succeeded. This is worth the extra write latency because the cost of being wrong (double-spending, incorrect balances) is unacceptable in a financial system.

Contrast that with Swiggy's or Zomato's "X people ordered from this restaurant today" counter, or a Flipkart product's view count. These are prime candidates for eventual consistency: propagating the exact number to every replica within, say, five seconds is entirely acceptable, and demanding strong consistency here would only slow down every single page load for no real user benefit.

Careem and Talabat's live driver/rider location tracking sits in an interesting middle zone — users tolerate a location being a second or two stale (eventual consistency is fine), but the underlying order or trip *status* transition (accepted, picked up, delivered) usually needs read-your-writes or stronger guarantees, since a driver and customer disagreeing about whether an order was marked delivered creates real support and trust problems.

3. Diagram

Strong consistency (quorum write):

```
Client -- write --> [Primary] --waits for ack--> [Replica-1]
                                   --waits for ack--> [Replica-2]
Client gets "OK" only AFTER a majority of replicas confirm.
Any read, anywhere, after this point sees the new value.
```

Eventual consistency (fire-and-forget):

```
Client -- write --> [Primary] --acks immediately--> Client gets "OK"
                                   |
                                   +-----(async, in background)----> [Replica-1]
                                   +-----(async, in background)----> [Replica-2]
A read hitting Replica-1 a moment later may still see the OLD value.
```

4. Trade-offs Table

Model	Guarantee	Cost	Good fit
Strong consistency	Always see latest write	Higher write latency, lower availability during partitions	Payments, ledgers, inventory counts
Eventual consistency	Converges 'eventually', no bound	Possible stale reads	Like counts, recommendations, view counts
Read-your-writes	You always see your own recent writes	Extra routing logic (pin reads to primary post-write)	User profile edits, comment posting
Causal consistency	Related events stay in order for everyone	More complex to implement than eventual	Comment threads, chat message ordering

5. Common Interview Questions

Q1: "Would you use strong or eventual consistency for a bank account balance? Why?"

Model approach: Strong consistency, unambiguously — explain that showing two different balances to two concurrent reads (or allowing a withdrawal to succeed against a stale balance that doesn't reflect a very recent deposit or another withdrawal) creates real financial and trust damage that outweighs the extra write latency strong consistency costs.

Q2: "A user posts a comment and it doesn't appear when they refresh immediately. What's happening and how do you fix it?"

Model approach: Identify this as a read-your-writes violation caused by the refresh hitting a lagging replica. Fix by routing that user's immediate post-write reads to the primary (or a replica known to be caught up), or by having the client optimistically render the just-posted comment locally without waiting for a fresh read at all.

Q3: "Give an example of causal consistency mattering in a real feature."

Model approach: A chat or comment-reply feature — if a reply can be seen before the message it replies to (because they landed on different, unsynchronized replicas), the conversation looks broken and confusing, even though neither message is individually 'wrong.' Causal consistency guarantees this specific ordering without paying for full strong consistency everywhere.

Q4: "Why not just make every field in your system strongly consistent, to be safe?"

Model approach: Explain that strong consistency isn't free — every write pays coordination latency and the system loses availability during partitions for that data. Applying it universally taxes high-volume, low-stakes fields (counters, recommendation caches) for no real benefit, when the correct engineering move is choosing the model per field based on actual business impact of staleness.

6. Practice Exercises

- 1 List five fields/features in a food delivery app (like Swiggy) and classify each as needing strong, eventual, or read-your-writes consistency, with a one-line reason.

- 2 Explain, in your own words, the difference between eventual consistency and causal consistency, using a chat app example for each.
- 3 A teammate proposes making the entire product catalog strongly consistent 'to keep things simple.' Write a short argument for why this is the wrong default, citing at least one concrete cost.

7. Curated YouTube Videos

Language	Video & Channel	Why it helps
English	"Consistency Models System Design Distributed Systems"	Covers strong, eventual, and causal consistency with clear definitions.
English	"Consistency Models Distributed Systems System Design"	A second pass with different examples, useful for reinforcement.
Hindi	"Complete System Design Roadmap 2025 HLD & LLD" — CodeHelp	Places consistency models in the context of the full Hindi HLD roadmap.
Hindi	"System Design Complete Course 2026" (Hindi + English, 10+ hrs)	Dedicated module on consistency trade-offs in a bilingual format.

8. Key Takeaways

- A consistency model is the promise a distributed system makes about when a write becomes visible to reads.
- Strong consistency: every read sees the latest write, at the cost of write latency and partition-time availability.
- Eventual consistency: replicas converge over time with no fixed bound, favoring speed and availability.
- Read-your-writes and causal consistency are practical middle grounds solving specific real bugs.
- Consistency should be chosen per data field based on business impact, not applied uniformly across a system.
- Financial and inventory data almost always need strong consistency; counters and recommendations rarely do.
- This chapter sets up Chapter 19's CAP theorem, which formalizes exactly when you're forced to choose.